

Decodificador Morse mediante el desarrollo de una Máquina de Turing

Facultad de Ingeniería del Ejército - Gral. Div. Manuel Savio

Goñe Juan Ignacio, Requena Melina.

I. ABSTRACT

En este trabajo se presenta la implementación de una Máquina de Turing capaz de decodificar secuencias en Código Morse y transformarlas en caracteres alfanuméricos. Para ello, se formaliza el problema dentro del marco de la Teoría de la Computación y se define una MT con reglas de transición específicas que permiten interpretar las combinaciones de puntos y rayas. La implementación se lleva a cabo en un simulador gráfico que permite visualizar el comportamiento de la máquina en tiempo real. Se evalúa su desempeño tanto en entradas válidas como en aquellas ingresadas con errores, garantizando una respuesta adecuada en cada caso.

II. OBJETIVOS

- Diseñar una Máquina de Turing (MT) capaz de traducir sentencias de Código Morse a caracteres alfanuméricos.
- Implementar el modelo en un simulador gráfico que permita observar el comportamiento de la MT.
- Asegurar un correcto funcionamiento para sartas válidas y manejo de sartas que se ingresen incorrectamente

III. PALABRAS CLAVE

Alfabeto: Conjunto no vacío y finitos de símbolos.

Sartas: Sobre un alfabeto A, es la secuencia finita de símbolos que pertenecen a A.

Lenguajes: Conjunto de palabras formadas por símbolos de un alfabeto.

Gramáticas: Estructura del lenguaje que da origen a formas válidas del lenguaje. Están compuestas por: $G(V, \Sigma, P, S)$ donde: V: vocabulario terminal, Σ : vocabulario no terminal, P: conjunto de producciones, S: símbolo inicial.

Autómata: modelo matemático, que dada una entrada de símbolos salta a través de una serie de estados de acuerdo con una función de transición.

Máquina de Turing: Modelo matemático que posee una memoria infinita en forma de cinta, así como un cabezal que puede leer y procesar los símbolos en la cinta, y moverse en cualquier dirección a lo largo de la misma.

Decodificador: Sistema combinacional que convierte un código codificado en su forma original o en una representación comprensible.

IV. INTRODUCCIÓN

La **Teoría de la Computación** es una rama de la Informática y Matemática que se ocupa del estudio de los modelos de cómputo y sus capacidades. Esta disciplina tiene sus orígenes a principios del siglo XX, cuando matemáticos y filósofos comenzaron a explorar los límites de lo computable. Alan Turing, Kurt Gödel y Alonzo Church son algunos de los principales precursores de la teoría moderna de la computación. La cuestión central de la teoría de la computación es entender qué puede ser resuelto por una máquina computacional, en qué condiciones y con qué recursos.

Uno de los avances más significativos en este campo fue la formulación de la Tesis de Church-Turing, que estableció una simbiosis entre el concepto de Máquina de Turing y el de función computable. Según esta tesis:

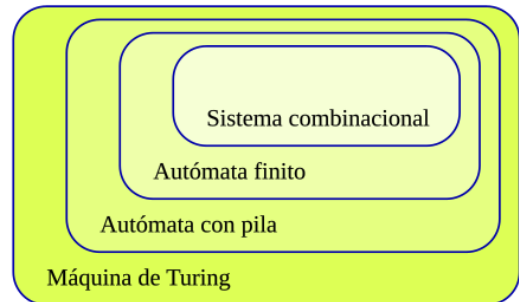
‘Todo algoritmo computable puede ser resuelto mediante una Máquina de Turing’.

Este principio no solo demostró que ciertos problemas son decidibles, sino también que existen problemas indecidibles, aquellos para los que no existe ningún algoritmo que pueda ofrecer una solución en tiempo finito.

A. La Jerarquía de Chomsky

Uno de los desarrollos más importantes en la teoría de la computación fue la clasificación de los lenguajes formales propuesta por Noam Chomsky en 1956, conocida como la Jerarquía de Chomsky. Esta clasificación divide los lenguajes formales, según los autómatas capaces de reconocerlos y las gramáticas que los generan. La jerarquía de Chomsky está formada por los siguientes niveles:

Teoría de autómatas



1) Lenguajes Tipo 3, Regulares:

Los **lenguajes regulares** son los más simples dentro de la Jerarquía de Chomsky y pueden ser generados por **gramáticas regulares**. Estas gramáticas son procesadas por **autómatas finitos**, los cuales tienen una estructura bien definida y restricciones claras. Los lenguajes regulares son fundamentales en la computación, ya que sirven como base para el diseño de **expresiones regulares**, utilizadas en tareas de búsqueda, análisis de patrones y reconocimiento léxico en compiladores.

Gramáticas Regulares

Las **reglas de producción** en una gramática regular tienen el siguiente formato:

$$A \rightarrow aB$$

$$A \rightarrow a$$

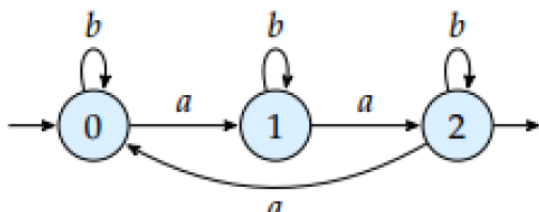
$$A \rightarrow \lambda$$

Donde:

- A y B son símbolos no terminales (variables).
- a es un símbolo terminal perteneciente al alfabeto del lenguaje.
- B puede existir o no en la producción.

Autómatas Finitos

Un **autómata finito** es un modelo matemático que representa un sistema capaz de reconocer cadenas (sartas) y determinar si pertenecen o no a un lenguaje.



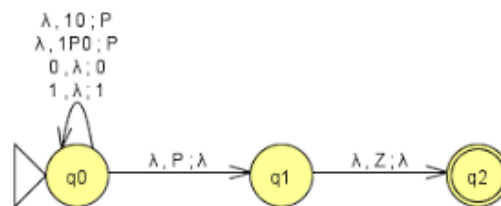
Expresiones regulares

Las expresiones regulares son una herramienta fundamental en la teoría de lenguajes formales y la computación. Fueron introducidas por Stephen Kleene en 1956 como un metalenguaje para describir los lenguajes regulares, es decir, permiten expresar el conjunto de palabras aceptadas por los autómatas finitos de manera compacta y algebraica.

2) Lenguajes Tipo 2, Libres de Contexto (CFL):

Los lenguajes libres de contexto (CFL, por sus siglas en inglés Context-Free Languages) forman la segunda categoría dentro de la Jerarquía de Chomsky. Son más expresivos que los lenguajes regulares (Tipo 3) y pueden describir estructuras más complejas.

Estos lenguajes son generados por **gramáticas libres de contexto** (CFG, Context-Free Grammars), y se representan mediante **autómatas de pila** (PDA, Pushdown Automata), que cuentan con una memoria adicional en forma de pila. Esta característica permite el manejo de estructuras recursivas, como las que encontramos en muchos lenguajes de programación.



Una **gramática libre de contexto** es una cuádrupla:

$$G=(V, \Sigma, P, S)$$

donde:

- V : Conjunto finito de símbolos no terminales.
- Σ : Conjunto finito de símbolos terminales (alfabeto del lenguaje).
- P : Conjunto de reglas de producción de la forma:

$$A \rightarrow \alpha A$$

$A \in V$ (un único símbolo no terminal).

α es una cadena (posiblemente vacía) de símbolos terminales y/o no terminales.

$S \in V$: Símbolo inicial de la gramática.

Autómatas de Pila (PDA, Pushdown Automata)

Los **lenguajes libres de contexto** pueden ser reconocidos por **autómatas de pila**, que son una extensión de los **autómatas finitos** pero con una **pila** como memoria auxiliar.

Un **autómata de pila** se define como una **séptupla**:

$$M=(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

donde:

- Q : Conjunto finito de estados.
- Σ : Alfabeto de entrada.
- Γ : Alfabeto de la pila
- $\delta: Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \rightarrow Q \times \Gamma^*$: Función de transición, que depende del estado actual, el símbolo de entrada (o ϵ) y el tope de la pila.

Como resultado, cambia el estado y puede modificar la pila.

- $q_0 \in Q$: Estado inicial.
- $Z_0 \in \Gamma$: Símbolo inicial de la pila.
- $F \subseteq Q$: Conjunto de estados de aceptación.

3) *Lenguajes Tipo 1, Sensibles al contexto:*

Los lenguajes sensibles al contexto (Context-Sensitive Languages, CSL) forman parte del Tipo 1 en la Jerarquía de Chomsky. Son generados por **gramáticas sensibles al contexto** (Context-Sensitive Grammars, CSG) y reconocidos por **autómatas de pila lineales** (Linear Bounded Automata, LBA).

Gramáticas Sensibles al Contexto (CSG)

Las gramáticas sensibles al contexto son una extensión de las gramáticas libres de contexto, en las cuales las reglas de producción dependen del contexto en el que aparece un símbolo.

Las producciones en estas gramáticas tienen la forma:

$$\alpha A \beta \rightarrow \alpha \gamma \beta$$

donde:

- A es un símbolo no terminal.
- α, β, γ son cadenas de símbolos terminales y/o no terminales.
- γ debe ser al menos tan larga como A, lo que impide la contracción de la cadena y garantiza que la longitud de la cadena nunca disminuya.

Autómatas de Pila Lineales (LBA)

Los **lenguajes sensibles al contexto** son reconocidos por autómatas de pila lineales, un tipo especial de **Máquina de Turing** con memoria limitada.

Un LBA es una **Máquina de Turing acotada**, lo que significa que:

1. Tiene una cinta cuya longitud es proporcional a la longitud de la entrada.

2. No puede moverse fuera de los límites impuestos por la entrada.

Se define como una séptupla:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_a, q_r)$$

donde:

- Q : Conjunto finito de estados.
- Σ : Alfabeto de entrada.
- Γ : Alfabeto de la cinta.
- δ : Función de transición que define cómo el autómata cambia de estado y modifica la cinta.
- q_0 : Estado inicial.
- q_a : Estado de aceptación.
- q_r : Estado de rechazo.

4) *Lenguajes Tipo 0, Sin restricciones*

Los lenguajes Tipo 0, también llamados lenguajes recursivamente enumerables, forman la clase más general y poderosa en la Jerarquía de Chomsky. Estos lenguajes son reconocidos por **Máquinas de Turing** (MT), cualquier problema computable puede representarse con un lenguaje Tipo 0.

Gramáticas Sin Restricciones

Las gramáticas Tipo 0 no imponen restricciones en sus reglas de producción, lo que les otorga la máxima flexibilidad para describir lenguajes. Una **gramática sin restricciones** es una cuaterna de la forma:

$$G = (V, \Sigma, P, S)$$

donde:

- V : Conjunto finito de variables (no terminales).
- Σ : Conjunto finito de símbolos terminales (alfabeto del lenguaje).
- P : Conjunto finito de reglas de producción.
- S : Símbolo inicial.

Las reglas de producción tienen la forma general:

$$\alpha \rightarrow \beta$$

donde:

- α y β son cadenas de símbolos terminales y/o no terminales.
- $\alpha \neq \varepsilon$

B. La Máquina de Turing

La Máquina de Turing es un modelo abstracto y teórico de cómputo que fue desarrollado por Alan Turing en 1936, con el objetivo de formalizar el concepto de “cálculo” de una manera precisa. Consiste en una cinta infinita dividida en celdas, donde cada celda puede contener un símbolo de un alfabeto finito. La máquina tiene un cabezal de lectura/escritura que se mueve a lo largo de la cinta, lee el símbolo en la celda actual, y luego realiza una acción según un conjunto de reglas predefinidas.

A pesar de su simplicidad, la Máquina de Turing es un modelo universal de cómputo: cualquier proceso computacional que pueda ser descrito por un algoritmo puede ser simulado por una Máquina de Turing.

Una **Máquina de Turing (MT)** se define formalmente como una **séptupla**:

$$M = \{Q, \Sigma, \Gamma, \delta, q_0, B, F\}$$

donde:

- Q : Conjunto finito de estados.
- Σ : Alfabeto de entrada.
- Γ : Alfabeto de la cinta (conjunto completo de símbolos de la cinta, donde Σ es un subconjunto de Γ).
- $\delta: q, X \rightarrow qn, S, D$: Función de transición, que define cómo la máquina cambia al siguiente estado,

modifica la cinta y se mueve a la izquierda o derecha.

- q_0 : Estado inicial.
- B : símbolo blanco, pertenece a Γ .
- F : conjunto de estados de aceptación, subconjunto de Q .

El propósito de este proyecto es implementar un decodificador Morse basado en una Máquina de Turing, lo que permitirá demostrar la capacidad de este modelo teórico para realizar una tarea que se encuentra dentro de sus capacidades computacionales.

C. Código Morse

El código Morse es un sistema de codificación que utiliza secuencias de señales, comúnmente representadas por puntos y rayas, para representar letras, números y otros caracteres. Fue inventado por Samuel Morse y Alfred Vail en 1836, y se utilizó principalmente en la telegrafía para transmitir información a través de señales eléctricas. Aunque en su origen se usaba para transmitir mensajes mediante sonidos, el código también puede ser representado visualmente, mediante luz o cualquier otro tipo de señal, como toques o destellos.

A · —	N — ·	1 · — — —	? · · — — · ·
B — · · ·	O — — —	2 · · — — —	! — · — — — —
C — · — ·	P — · — ·	3 · · · — —	· — · — — —
D — · ·	Q — — — —	4 · · · —	, — · — — —
E ·	R — ·	5 · · · ·	; — · — — ·
F · · — ·	S · ·	6 — · · ·	: — — — · ·
G — — ·	T —	7 — — · ·	+ — · — ·
H · · · —	U — ·	8 — — · ·	- — · · · —
I · ·	V · · —	9 — — — ·	/ — · — ·
J — — — ·	W — — —	0 — — — —	= — · · — —
K — · —	X — · —		
L — · —	Y — — —		
M — —	Z — — ·		

V. DESARROLLO DE CONTENIDOS

A. Funcionamiento del decodificador morse

El decodificador tiene como objetivo realizar la traducción de código Morse a texto legible de manera eficiente. Su funcionamiento se basa en procesar individualmente cada carácter ingresado en código Morse, traducirlo a su equivalente en texto alfanumérico y, al finalizar el ingreso de todos los caracteres, mostrar en la cinta la cadena completa de texto traducido. Se decidió implementar este decodificador en una máquina de Turing, ya que en un autómata finito o de tipo pila, habría sido posible reconocer un mensaje de código Morse válido, pero solo la cinta de la máquina de Turing otorga la posibilidad de escribir el mensaje decodificado. La máquina de Turing deberá seguir estos pasos:

- Leer y reconocer secuencias de símbolos Morse: La máquina deberá ser capaz de leer secuencias de puntos (.) y rayas (_) en la cinta, interpretándolas de acuerdo con el alfabeto de código Morse.
- Asignar los caracteres correspondientes: Para cada secuencia Morse reconocida, la máquina debe asociar el carácter correspondiente (letras, números o signos de puntuación).

A continuación, se detallan los pasos del funcionamiento del decodificador diseñado:

Inicialización de la cinta:

Al iniciar la máquina, en la cinta debe encontrarse un mensaje en código morse, que la cinta traducirá al alfabeto latino. Este mensaje debe estar representado con puntos (.), rayas (_), y espacios (-), y debe encontrarse al final un carácter de finalización de mensaje (#).

Traducción del mensaje:

A medida que el cabezal avanza sobre la cinta, procesa la secuencia de puntos y rayas, cambiando de estado según las combinaciones encontradas. Al leer un espacio, lo sustituye por el carácter correspondiente en el alfabeto latino.

Finalización:

Cuando la máquina encuentra el carácter de finalización (#), la sarta es aceptada. Si el código Morse no es válido, la máquina entrará en un estado de falla antes de alcanzar el estado de aceptación.

B. Diseño de la Máquina de Turing

Para llevar a cabo este análisis, definiremos formalmente la *MT DECODIFICADOR*, como

$M = \{Q, \Sigma, \Gamma, \delta, q_0, B, F\}$, donde:

- Q (conjunto finito de estados) = $\{q_0, q_1, \dots, q_{99}, q_{100}\}$.
- Σ (conjunto finito de símbolos de entrada) = $\{., -, \# \}$.
- Γ (conjunto completo de símbolos de la cinta, donde Σ es un subconjunto de Γ) = $\{a, b, \dots, z, 0, 1, 2, 3, \dots, 9, -, _ , . , ? , ! , , , ' , : , + , / , = \}$.
- δ (función de transición. $\delta(q, X)$, con un estado q y un símbolo X)
- δ define (q_n, S, D) ,
- q_n : siguiente estado
- S : símbolo escrito en la cinta
- $D = \{L, R, H\}$, sentido del movimiento del cabezal.
- B símbolo blanco, pertenece a Γ .
- F (conjunto de estados de aceptación, subconjunto de Q) = $\{q_4\}$.

C. Ejemplo práctico del funcionamiento

Entrada

Al iniciar la máquina, la cinta contiene la siguiente sarta:

. . _ . - . . - . - #

Traducción

.._. traducido a F

La máquina detecta un punto, y se mueve del estado q0 al q1

Luego, estando en q1, la máquina lee otro punto, y se mueve al estado q6

El siguiente carácter es una raya, por lo que de q6 el autómata pasará a q20

Desde q20, lee otro punto, por lo que se mueve a q22

Finalmente, desde el estado q22, detecta un espacio (-), por lo que el carácter se terminó de escribir, y puede pasar al estado q23, que reemplaza el espacio por el carácter correspondiente al camino recorrido hasta ahora

Cinta después de la traducción:

.._. F .. - .- #

Como el siguiente símbolo leído no es un #, la máquina intentará traducir el próximo carácter.

.. traducido a I

Cinta después de la traducción:

.._. F .. I .- #

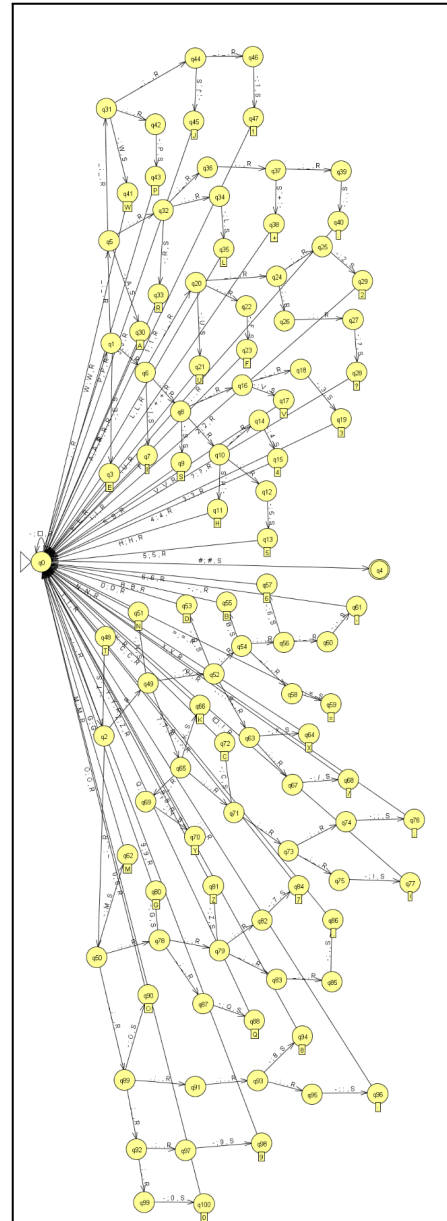
. traducido a E

Cinta después de la traducción:

.._. F .. I . E #

Cinta final:

.._. F .. I . E



VI. SIMULACIÓN

- Todos los caracteres

.._. F .. I . E #

